

ENTERPRISE COMPUTING

Virtualization & Containerization

LAB 4/W5

4TH YEAR - ENSIA

OBJECTIVES

- Explain core Spring Boot concepts and auto-configuration
- Containerize a Spring Boot app & Deploy multi-service architecture
- Scale services with Docker Swarm
- Use object storage

DEMONSTRATION OF LAB3 BY STUDENTS

Each student needs to demonstrate that they have completed fully lab 3 by demonstrating access to the access endpoints to a gateway + orchestration service.

INDIVIDUAL EXERCISES

Task / Exercise C1 [5 Minutes] (Mastering Java Spring Boot + Architecture) :

Spring Boot Questions:

1. What problem does Spring Boot solve compared to traditional Spring Framework?
2. What are the key components of the Spring Boot framework?
3. What's the purpose of `@SpringBootApplication`, `@RestController`, `@Service`, `@Repository`?
4. What is Spring Boot Actuator?
5. Why should configuration not be hardcoded ?

Architecture Questions:

1. Why were Documents and Comments implemented as separate services?
2. What architectural problem does the Gateway solve?
3. Why is orchestration centralized in the ESB service?

Task / Exercise C2 [30 Minutes] (Hello World Docker)

Part 1: Dockerfile

1. Within your project from prev lab, create a file named : **Dockerfile** with the following content (Change the path based on Maven/Gradle..):

```
FROM eclipse-temurin:21-jre-jammy
WORKDIR /app
COPY build/libs/documents-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

2. Build and run via executing the following command :

```
docker build -t dms-documents .
docker run -p 8081:8081 dms-documents
```

3. Test using Postman or a browser
4. Do the same for the Comments Service.

Part 2: Docker Compose

Deploy the following multiple services together : Documents Service, Comments Service Gateway Service (You need to change application.yml for case of **localhost**)

1. Make sure to create a Dockerfile for each service.
2. Under a separate folder, create the docker-compose.yml file with the following :

```
version: "3.9"
services:
```

```

documents:
  image: dms-documents
  ports:
    - "8081:8081"
comments:
  image: dms-comments
  ports:
    - "8083:8083"
gateway:
  image: dms-gateway
  depends_on:
    - documents
    - comments
environment:
  DOCUMENTS_SERVICE_URL: http://documents:8081
  COMMENTS_SERVICE_URL: http://comments:8083
ports:
  - "8080:8080"

```

3. Run the services using the command : **docker compose up**

Task / Exercise C3 [40 Minutes] (Object Storage Integration) :

You are requested to upload a single file for each document being added to the document service. The file will be stored into an S3 storage engine MinIO.

Part 1 : MinIO Setup & Hello World

1. Setup a Docker for MinIO (Use the config lines below inside the compose file)

```

minio:
  image: "quay.io/minio/minio:latest"
  ports:
    - "9000:9000"
    - "9001:9001"
  command: ["server", "/data", "--console-address", ":9001"]
  environment:
    MINIO_ROOT_USER: "admin"
    MINIO_ROOT_PASSWORD: "ensia123456"
  volumes:
    - minio_data:/data
volumes:
  minio_data:

```

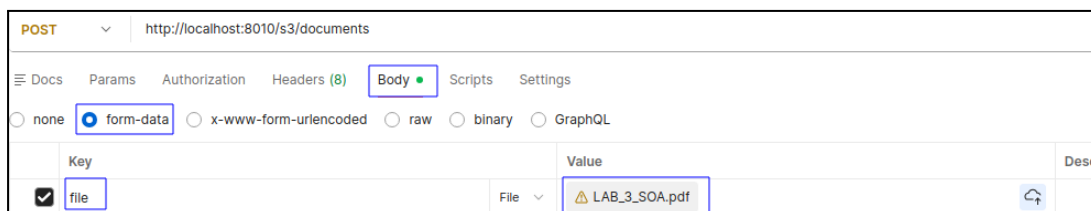
2. Create your first bucket and Upload manually a file to S3 using the web portal (<http://localhost:9001/>) (Example LAB5.pdf)
3. Create a new Java Spring Boot project that serves as the hello world to download the file from the S3 storage engine.
 - a. Make sure to put the credentials of S3 inside the **application.properties** as done in the Docker Compose file

```

spring.application.name=s3-hello-world
server.port=8010
app.s3.endpoint=${S3_ENDPOINT:http://localhost:9000}
app.s3.region=${S3_REGION:us-east-1}
app.s3.access-key=${S3_ACCESS_KEY:minioAccessKey}
app.s3.secret-key=${S3_SECRET_KEY:minioSecretKey}
app.s3.bucket=${S3_BUCKET:ensia}
app.s3.path-style-access=${S3_PATH_STYLE_ACCESS:true}
app.s3.presign-duration-seconds=${S3_PRESIGN_DURATION_SECONDS:300}

```

- b. Implement a simple Endpoint : s3/document/lab5 which gives the PDF file when access via a browser.
4. Within the hello world, add another API Endpoint to for uploading document



Part 2 : Integration with the DMS

5. Within your DMS solution, add the necessary endpoints/services to allow adding a document file when adding a document in addition to downloading the attached file.

Hello World Project is :

https://www.dropbox.com/scl/fo/xlxm677eoe0dxnp1lc1fk/ACjqDyn3bHVXTULyxpD9_jM?rlkey=y5xh311qzqx6sk1pqr1xitw18&st=cf45439x&dl=0

NEXT DELIVERABLE : 27 FEBRUARY 2025

The backend 1st increment for the Document Management System (DMS) must be completed individually to include the following :

- Use of Docker Swarm with the Integration of the following services.
 - Documents
 - Comments
 - ESB.
 - Gateway
 - Integration of S3 Storage.
- Use an External relational database engine to store the list of documents.

Students must also work on the following features

- Spring Boot Actuator
- Replication of Storage (Either S3 or Relational data)
- Restart policies

OPTIONAL EXERCISES

Task / Exercise P1 :

1. Why is containerization useful for microservices?
2. Does the container include the database?
3. What happens to stored data when the container is removed?
4. What is docker compose?
5. How does docker compose differ from Docker swarm?
6. Identify at least two Single Points of Failure in the current architecture.
7. If 100,000 users access the system simultaneously, what components will fail first?
8. How could this architecture be improved for high availability?